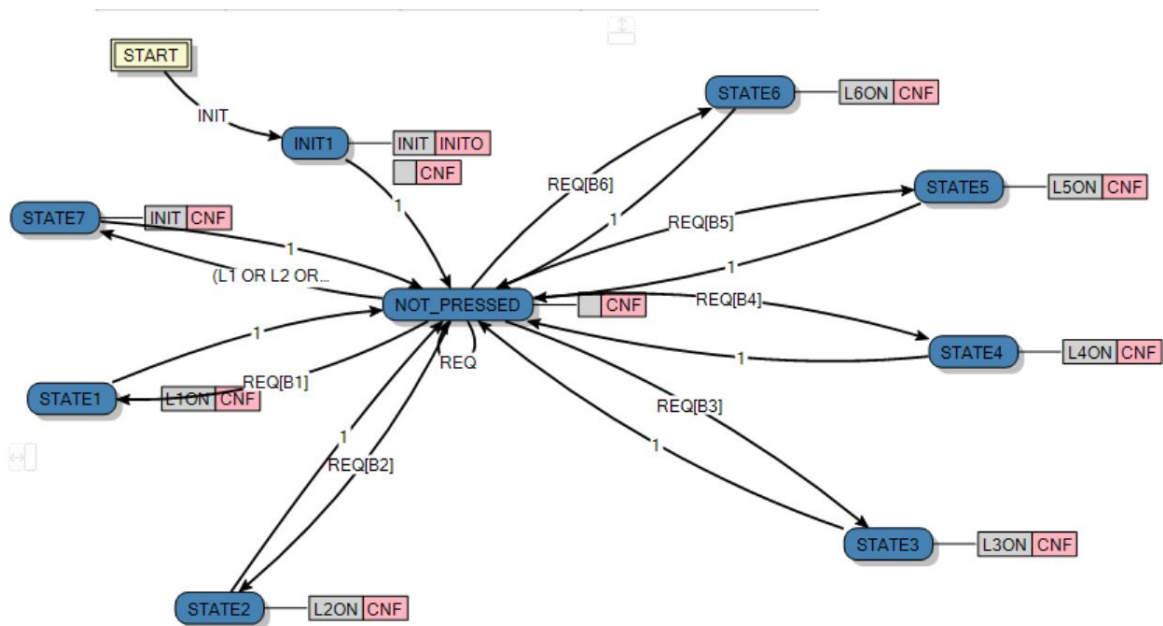


Assignment 2 Report

Anton Smedholm 907271

Btn_lock:



How it works:

After INIT event we execute INIT algorithm that set all L3 to FALSE, after that we proceed to NOT_PRESSED state.

NOT_PRESSED state has REQ transition to itself and triggers CNF to keep updating CabinControl. If ButtonX gets pressed, Bx becomes TRUE and ECC moves to STATEX. LXON algorithm gets triggered where LX gets TRUE and others FALSE.

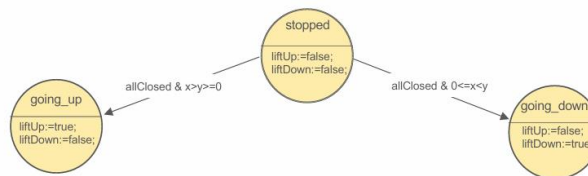
If some of LX is on and there is input event RESET all LX get value FALSE and we return to NOT_PRESSED.

CabinControl:

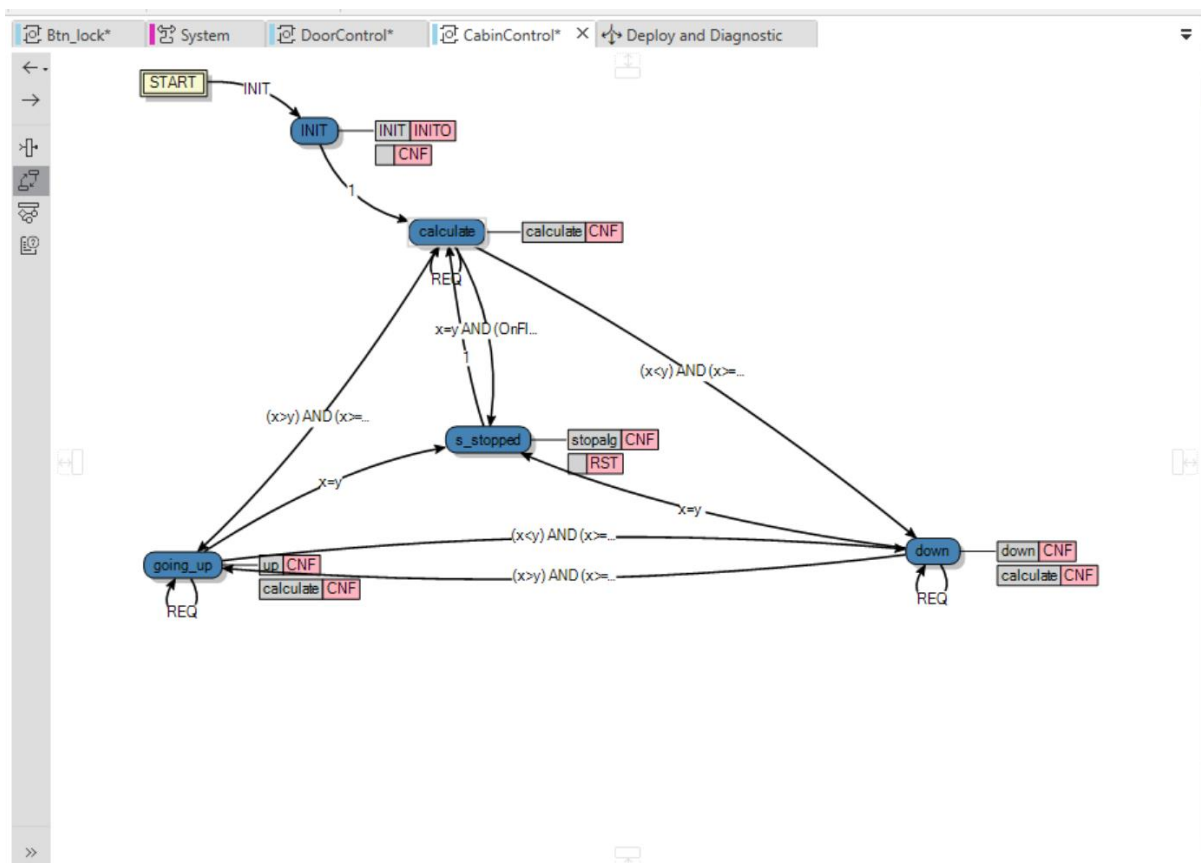
State machine from lecture:

Moving between floors: add transitions

allClosed: BOOL	all doors closed	x: int	where elevator is called	y: int	current floor
<pre>allClosed := DoorClosed0 AND DoorClosed1 AND DoorClosed2;</pre>		<pre>IF ButtonAt0 OR ButtonFor0 THEN x:= 0; ELSIF ButtonAt1 OR ButtonFor1 THEN x:= 1; ELSIF ButtonAt2 OR ButtonFor2 THEN x:= 2; ELSE x:=-2; END_IF;</pre>		<pre>IF OnFloor0 THEN y:=0; ELSIF OnFloor1 THEN y:=1; ELSIF OnFloor2 THEN y:=2; ELSE y:=-2; END_IF;</pre>	



ECC:

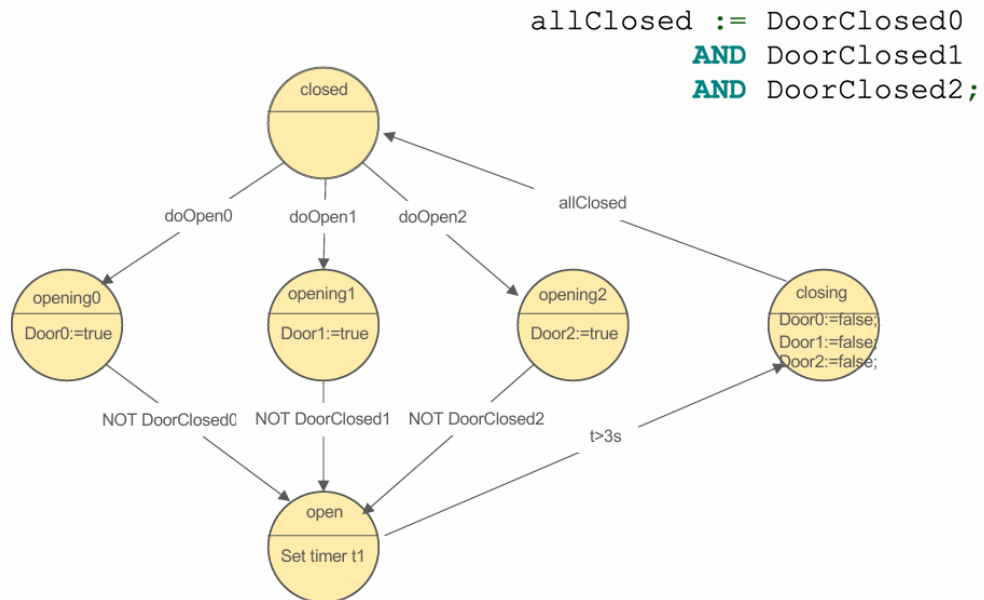


My ECC is based on lecture's state machine, but in my version it is possible to change direction without stopping. The stopped state was separated in calculate and s_stopped. S_stopped makes elevator to stop and turns buttons' lights off using RST output event. Calculate state is used to execute calculate algorithm which sets value for y based on which of OnFloorY variables is TRUE and sets value to x according which LX is TRUE.

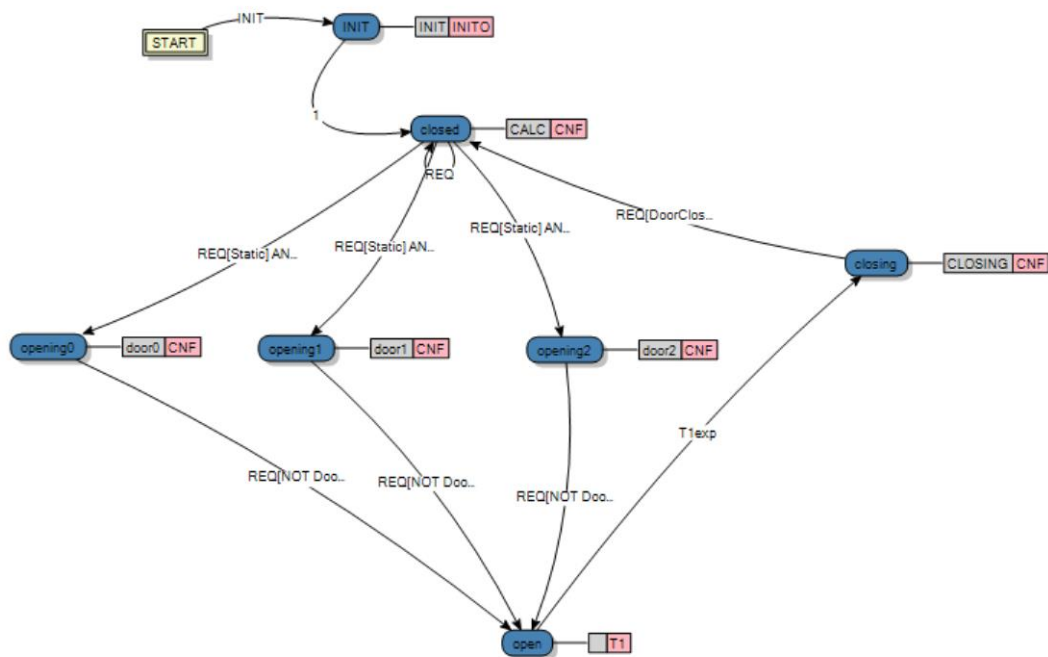
IF x is bigger than y it transits to going_up state where up algorithm makes elevator to go up. Calculate algorithm here also update x and y until x=y and elevator stops or direction of elevator need to be changed. Same with down state.

DoorControl:

State machine from lecture:



ECC:



My ECC is same as lecture's statemachine. Instead of doOpenX I used REQ[Static] **AND** (REQ[ButtonForX] **OR** REQ[ButtonAtX]) **AND** REQ[floor=X] for transition to opening. OpeningX sets doorX TRUE and allClosedFalse. I used same transition trigger to open as in lecture.

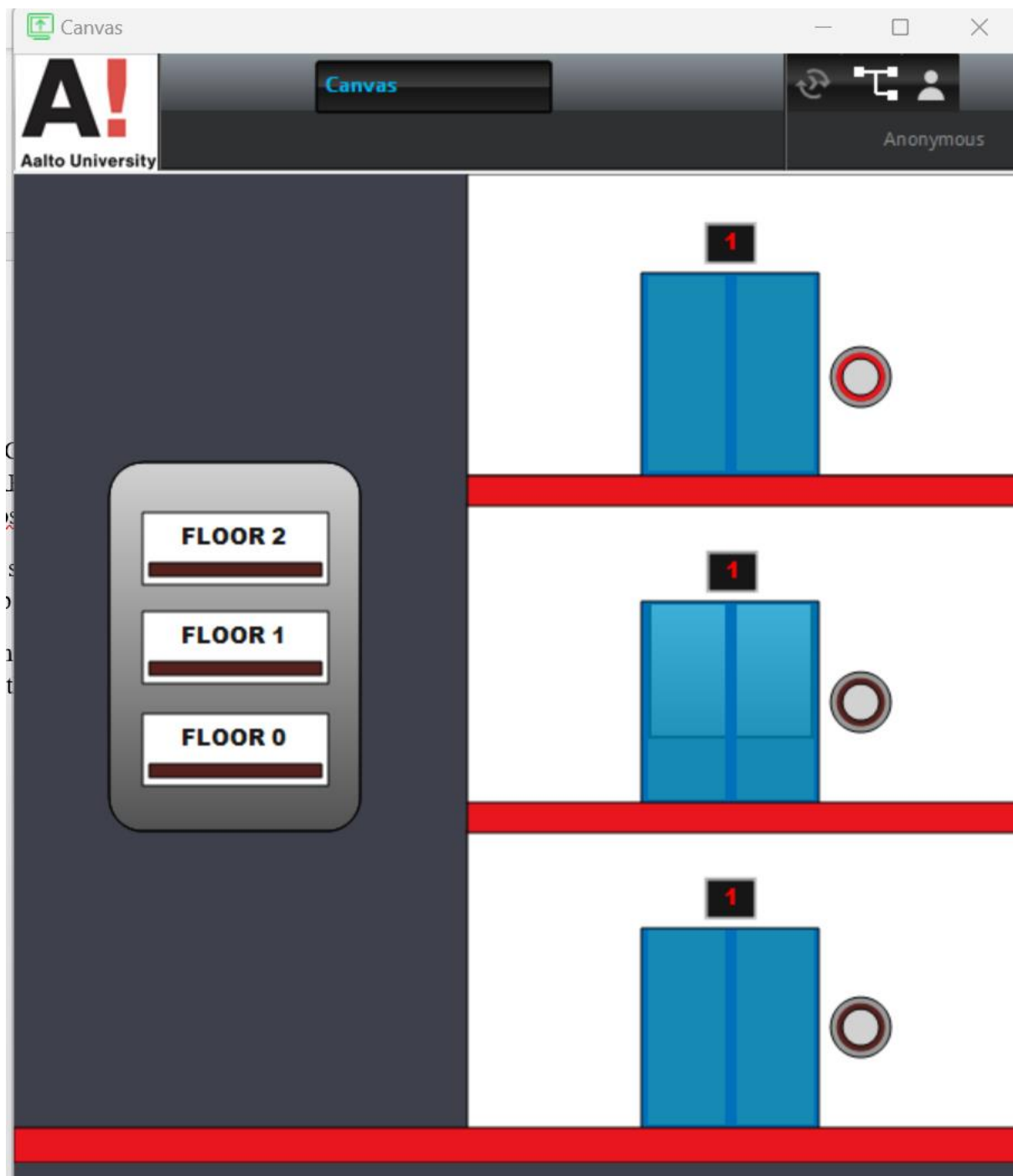
Open state triggers T1 output event that launches E_DELAY with 3000ms. When E_DELAY is out it trigger T1exp event and we transit to closing.

Closing sets all doorX to FALSE. To transit to closed we wait for all DoorClosedX to be true. In closed CALC algorithm set allClosed to TRUE.

Mapping:

I mapped DoorControl and E_DELAY to DEV1 and other blocks to DEV0.

Evidence of the system's correct behaviour:



Test where done using debugger and HMI.

First I checked “Btn_lock: reacts on button presses and sets the lights behind the buttons accordingly. The requirement is that the light can be only behind one of the six buttons. The light clears when the requested action is completed.”

When I press button in HMI it lights up. If I press another button, the first one turns off. When I push one button at elevator stops on the floor of this button, all lights clear.

Then “

The desired behaviour is the following: when the elevator is called by either button (inside and outside calls have equal priorities), it moves to the closest requested floor, opens the doors, and closes them after three seconds. Note that in this case starvation is possible. For example, let the elevator be located at floor 2 and called to floor 0. It starts moving and immediately it is called to floor 1. Then it is called again to floor 2. If requests for floors 2 and 1 keep being issued, the elevator will never get to floor 0.”

I call elevator from floor 2 to floor 0. It starts doing down and I call it to floor 1. It stops on floor 1 and opens door for 3s. It will stay on floor 1 if neither of buttons is pressed. If I call it to floor 2 and on half way to floor 1, it will not stop on floor 2. It will go from floor 2 to floor 1 without opening doors on floor 2 and it will open door when it will stop on floor 1.

This behaviour satisfies all requirements and is not against them.

Conclusion:

I wasted 12 hours on this assignment to make it works as written in instruction. It satisfies all requirements and I think it is worth full points.